Fibonacci Series –

Space Complexity (Detailed Analysis)

Program:

```
// Fibonacci Series through Recursion

// This code prints the Fibonacci series using recursion.
The Fibonacci series is a series of numbers where the next
number is the sum of the previous two numbers. The first two
numbers of the series are 0 and 1. The series is as follows:
0, 1, 1, 2, 3, 5, 8, 13, 21, 34, etc.

#include <iostream>

using namespace std;

int fibonacci(int n) // Function to calculate the nth
Fibonacci number
{
    if (n == 0 || n == 1) // Base case
    {
        return n; // If n is 0 or 1, return n
    }
    return fibonacci(n - 1) + fibonacci(n - 2); // Recursive
call : nth Fibonacci number is the sum of (n-1)th and (n-
2)th Fibonacci number
}

int main() // Main function
{
    int n;                                  // Number of
terms
    cout << "Enter the number of terms: "; // Input the
number of terms
    cin >> n;                               // Store the
number of terms
```

```
    cout << "The Fibonacci series is: ";   // Output the
Fibonacci series
    for (int i = 0; i < n; i++)           // Loop to print
the first n Fibonacci numbers
    {
        cout << fibonacci(i) << " "; // Print the ith
Fibonacci number
    }
    return 0; // Return statement
}
```

Space Complexity :

1. ***Input Space***

Input space refers to the memory allocated for the variables and parameters provided to run the algorithm, independent of the algorithm's internal execution steps.

- ***Holistic Input Space***: *In* $main$ *function, allocated memory for an integer* n *(the user input) and a loop counter* i.
  Because these are simple primitive types (usually 4 bytes each in $C++$*), they take a constant amount of memory regardless of how large the input number grows. Therefore, the holistic input space is* $\boldsymbol{O}(\mathbf{1})$.

- ***Parametric Space***: *In the* $fibonacci(int\ n)$ *function, the parameter* n *is passed by value. Every time the function is called, memory is allocated to hold this integer within the activation record (stack frame) of that specific call.*
  Because it is a single integer, the parametric space per call is $\boldsymbol{O}(\mathbf{1})$.

3. Auxiliary Space (VERY IMPORTANT PART)

Now the real complexity comes from recursion.

Understanding recursion stack

Each call to:

```
    int n;
    cout << "Enter the number of terms: ";
    cin >> n;
    cout << "The Fibonacci series is: ";
    for (int i = 0; i < n; i++)
    {
        cout << fibonacci(i) << " ";
    }
```

$creates\ a\ stack\ frame\ containing$:

- $parameter\ n$
- $return\ address$
- $temporary\ storage$

$Each\ frame\ =\ \Theta(1) space\ per\ recursive\ call$

Maximum recursion depth

Even though Fibonacci creates ***two recursive calls****, they are*

not simultaneous in stack*.*

<u>Conceptual clearance first</u>

1. ***Edges are the path between two nodes , but after***

execution of base returning **`0`** $\left(\boldsymbol{output\ value\ of\ leaf\ node}\begin{bmatrix}\boldsymbol{Base}\\ \boldsymbol{Case}\end{bmatrix}\right)$

is not a node but nodes are recursive call stack frames

from root to leaf node $[\boldsymbol{Here}\ , \boldsymbol{Fib}(\mathbf{0})\ \boldsymbol{and}\ \boldsymbol{Fib}(\mathbf{1})]$.

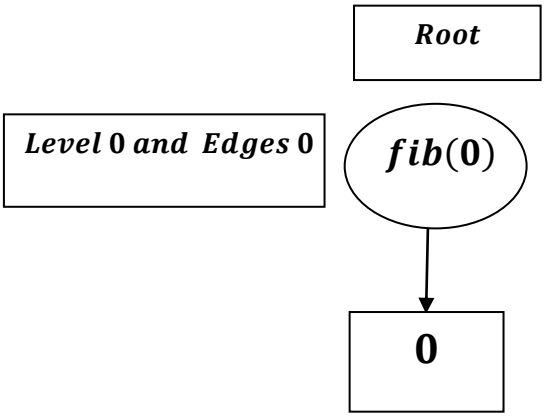
2. ***Levels are group of nodes at the same distance from the root****.*

3. ***Depth or Maximum Depth is the maximum edges from root***

node to leaf node*.*

Execution is depth − first*:*

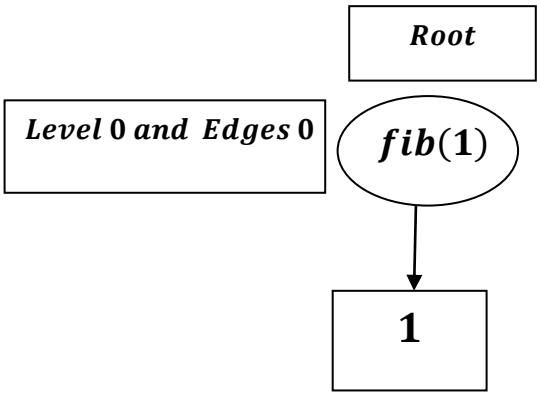
$1)\ When\ fibonacci(0)\ ,we\ get\ a\ single\ node\ i.e.:$



$Nodes = Edges + 1(Root) = 0 + 1 = 1[Here\ there\ is\ no\ edges]$

$Hence, Depth = 0.$

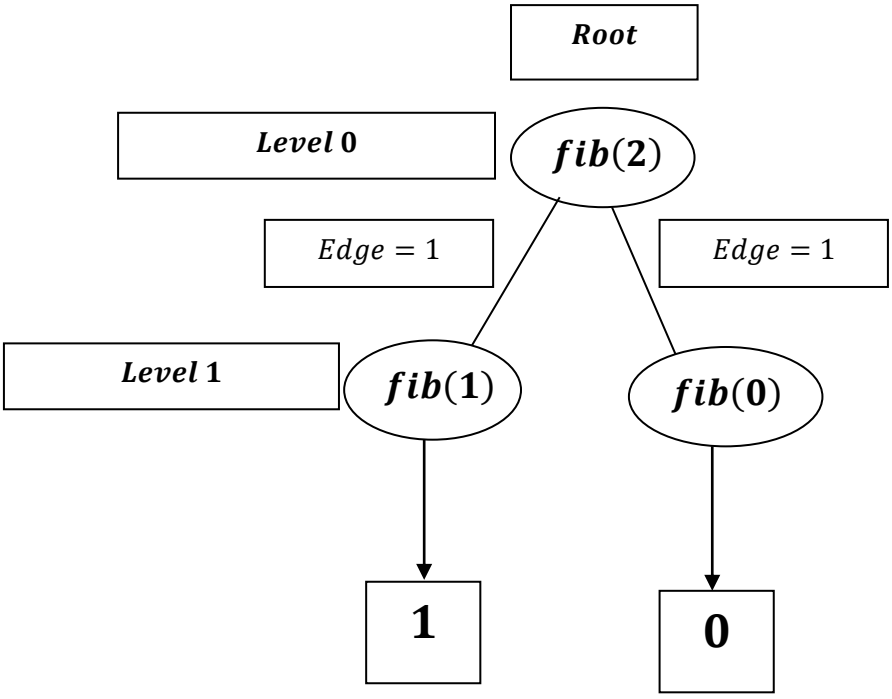
$2)\ When\ fibonacci(1)\ ,we\ get\ a\ single\ node\ i.e.:$



$Nodes = Edges + 1(Root) = 0 + 1 = 1[Here\ there\ is\ no\ edges]$

$\therefore Depth = 0$

$\mathbf{3)}\ \boldsymbol{When\ fibonacci(2)\ ,we\ get\ \ recursion\ tree}$:

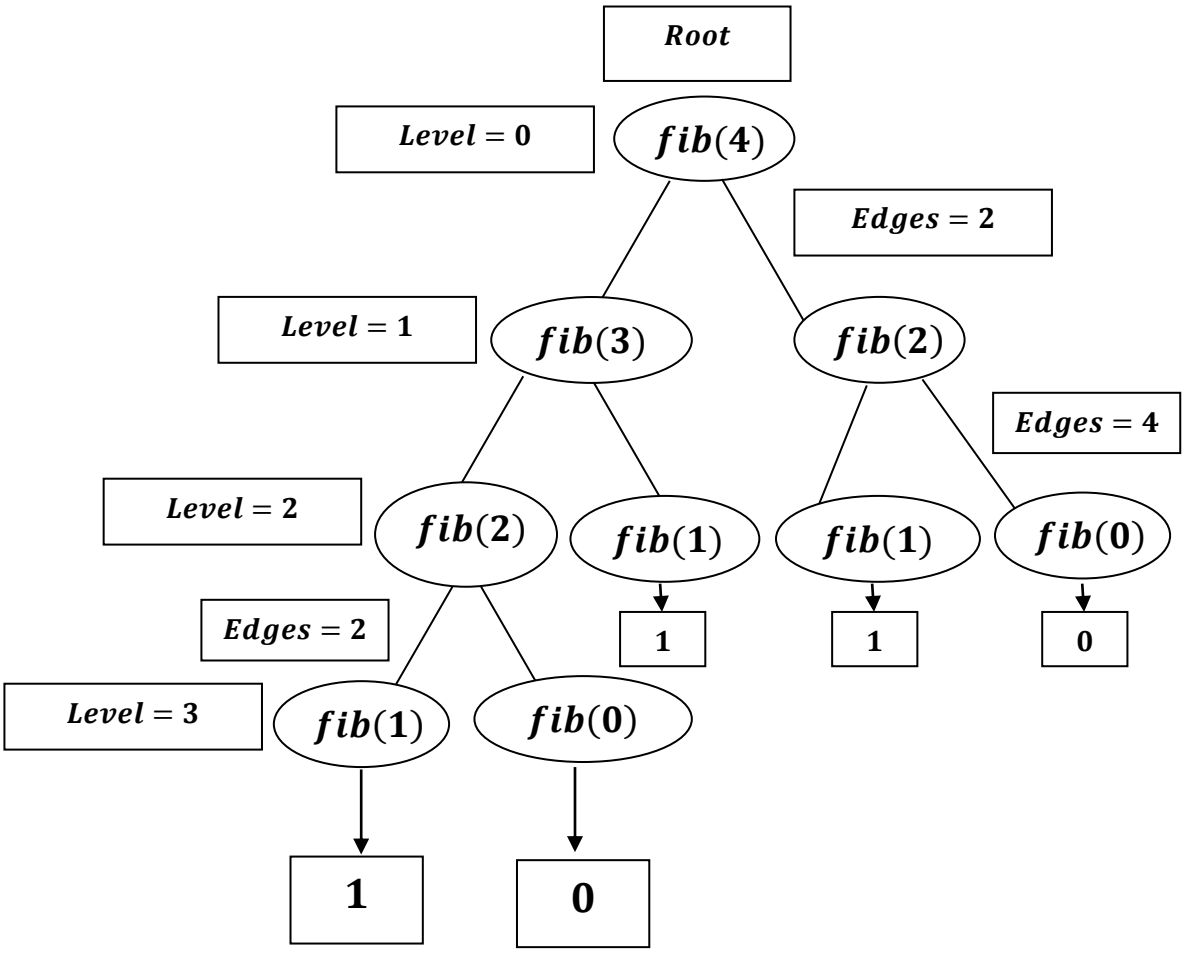


$$\boldsymbol{Nodes = Edges + 1(Root) = 2 + 1 = 3}$$

$\boldsymbol{But\ Depth\ \ = 1\ ,as\ maximum\ number\ of\ edges\ form}$

$\boldsymbol{Fib(2) to\ Fib(1)\ or\ Fib(2)\ to\ Fib(0) is\ 1}$.

4) $When\ fibonacci(4)\ ,we\ get\ recursion\ tree$:



$$Nodes = Edges + 1(Root) = 8[2 + 4 + 2] + 1 = 9$$

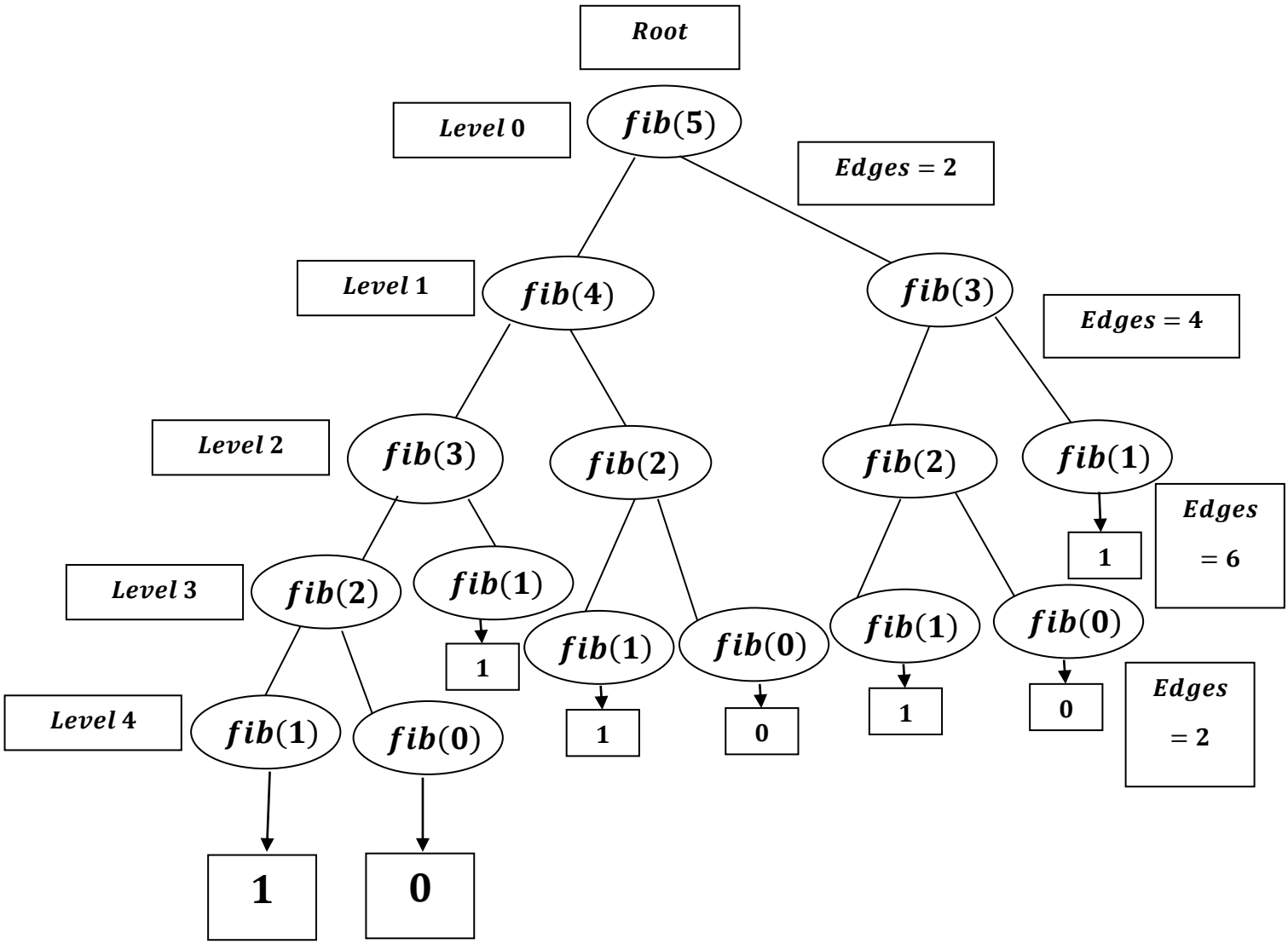
$Depth$:

1)$Fib(4) \rightarrow Fib(3) \rightarrow Fib(2) \rightarrow Fib(1) = Edges = 3$

2)$Fib(4) \rightarrow Fib(3) \rightarrow Fib(2) \rightarrow Fib(0) = Edges = 3$

$Therefore\ ,Depth = 3[Root\ to\ Leaf]$.

$In\ Addition: fibonacci(5)$, $we\ get\ recursion\ tree \rightarrow$



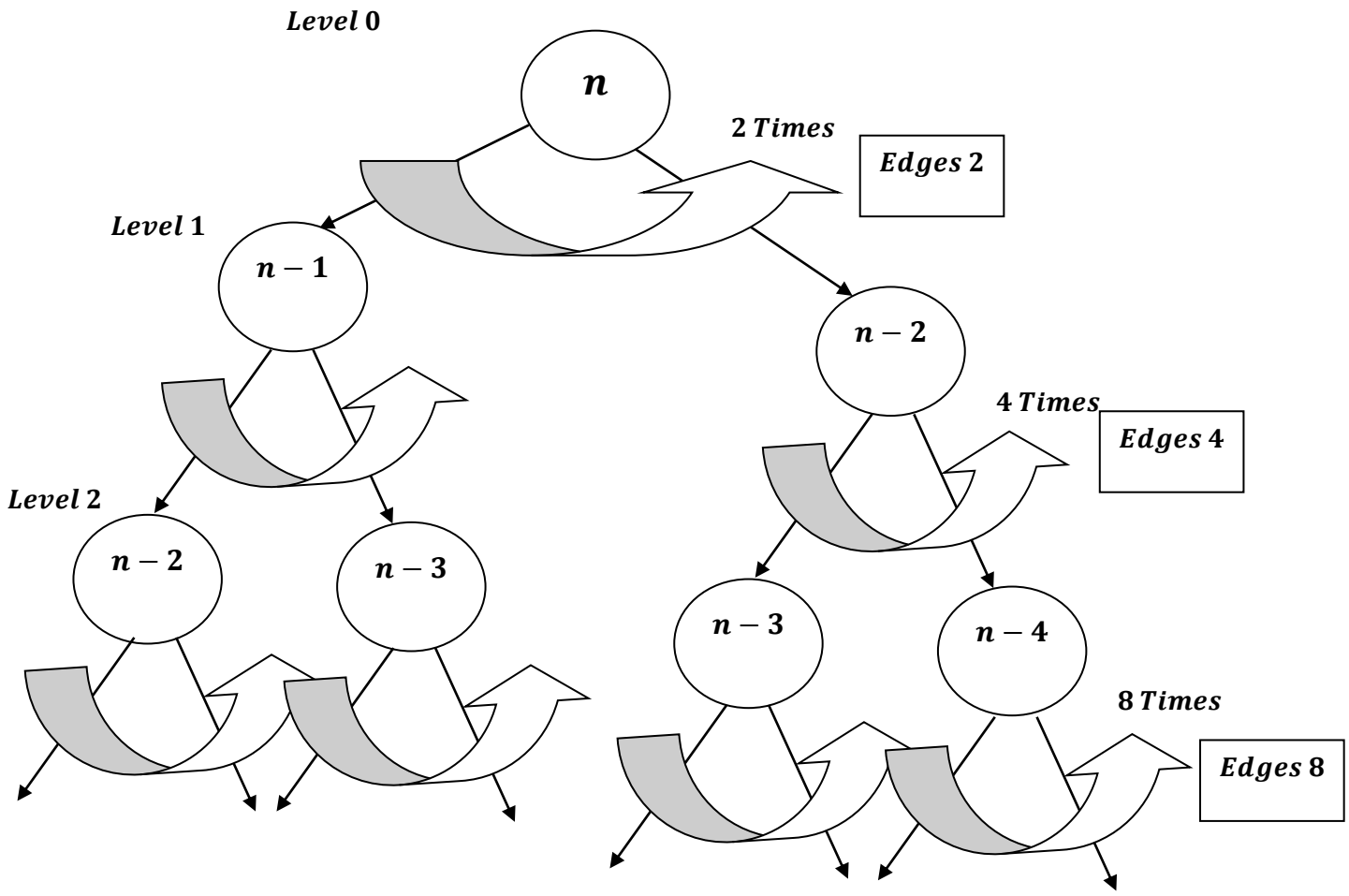
$$Nodes = Edges + 1(Root) = 14(2 + 4 + 6 + 2) + 1 = 15$$

$Depth$:

$1) Fib(5) \rightarrow Fib(4) \rightarrow Fib(3) \rightarrow Fib(2) \rightarrow Fib(1) = Edges = 4$

$2) Fib(5) \rightarrow Fib(4) \rightarrow Fib(3) \rightarrow Fib(2) \rightarrow Fib(0) = Edges = 4$

$Therefore\ , Depth = 4[Root\ to\ Leaf]$.



$The\ above\ tree\ to\ depict\ fibonacci\ series\ a\ perfect\ binary$

$binary\ tree\ , which\ is\ not\ in\ actual\ sense.$

$Maximum\ Number\ of\ Levels = k$

$As, fibonacci\ series\ is\ not\ perfect\ binary\ tree.$

$There\ for\ level\ k \leq 2^k\ nodes.$

$We\ know, Edges = Nodes - 1$

$Now, for\ `n`\ , say\ stopping\ at\ fib(1).$

$fib(n) \rightarrow fib(n-1) \rightarrow \cdots \rightarrow fib(1)$

$Then$:

$n - k = 1[user\ input\ `n` - Edges(k) = 1]$

$\Longrightarrow k = n - 1$

$Therefore\ , total\ nodes\ for\ level\ k =$

$2^0 + 2^1 + 2^2 + \cdots + 2^k$

$When, k = n - 1, then$:

$2^0 + 2^1 + 2^2 + \cdots + 2^{n-1}$

$By\ geometric\ series$:

$Geometric\ progression(series) upto `n` \sum_{i=0}^{n} ar^i$:

$$S(n) = ar^0 + ar^1 + ar^2 + \cdots + ar^n = a\left(\frac{r^{n+1} - 1}{r - 1}\right) , where$$

$S(n)\ is\ sum\ of\ first\ n\ terms\ , a\ is\ coefficient\ and$
$r\ is\ common\ ratio\ between\ the\ consecutive\ terms.$

$$\Rightarrow 1\left(\frac{2^{n-1+1}-1}{2-1}\right)$$

$$\Rightarrow 2^n - 1\ nodes.$$

$$\therefore Entire\ recursion\ tree\ will\ have \leq\ (2^n - 1) - 1\ edges.$$

$$Total\ number\ of\ Edges \leq 2^n - 2\ edges.$$

<u>***For Depth Calculation***</u>

$$But\ for\ longest\ path\ or\ \ depth, edge(k) = n - 1.$$

$$Again, for\ depth:$$

$$N(Nodes) = Edges(k) + 1(Root)$$

$$\Longrightarrow N = (n - 1) + 1 = n\ \ , then:$$

$$Maximum\ Depth = Number\ of\ Edges = n - 1.$$

$\boldsymbol{Based\ on\ Active\ stackframe}$:

$$\therefore \boldsymbol{Total\ Auxiliary\ Space}$$
$$= (\boldsymbol{Maximum\ Depth} + \mathbf{1}) \times \begin{pmatrix} \boldsymbol{Space\ per} \\ \boldsymbol{Recursive\ Call} \end{pmatrix}$$

$$\begin{bmatrix} \boldsymbol{Here} + \mathbf{1}\ \boldsymbol{for\ Root\ Node\ , which\ is\ also\ an\ active} \\ \boldsymbol{stack\ frame} \end{bmatrix}$$

$$= \boldsymbol{\Theta}(\mathbf{n} - \mathbf{1} + \mathbf{1}) \times \boldsymbol{\Theta}(\mathbf{1})$$
$$= \boldsymbol{\Theta}(\mathbf{n}) \times \boldsymbol{\Theta}(\mathbf{1})$$

$$= \boldsymbol{\Theta}(\boldsymbol{n})$$

$Now, for\ `n`\ , say\ stopping\ at\ fib(0).$

$fib(n) \rightarrow fib(n-1) \rightarrow \cdots \rightarrow fib(0)$

$Then$:

$n - k = 0[user\ input\ `n` - Edges(k) = 0]$

$\Longrightarrow k = n$

$Therefore\ , total\ nodes\ for\ level\ k =$

$2^0 + 2^1 + 2^2 + \cdots + 2^k$

$When, k = n, then$:

$2^0 + 2^1 + 2^2 + \cdots + 2^n$

$By\ geometric\ series$:

$$S(n) = ar^0 + ar^1 + ar^2 + \cdots + ar^n = a\left(\frac{r^{n+1} - 1}{r - 1}\right)\ , where$$

$S(n)\ is\ sum\ of\ first\ n\ terms\ , a\ is\ coefficient\ and$

$r\ is\ common\ ratio\ between\ the\ consecutive\ terms.$

$$\Rightarrow 1\left(\frac{2^{n+1}-1}{2-1}\right)$$

$$\Rightarrow 2^{n+1}-1\ nodes.$$

$$\therefore Entire\ recursion\ tree\ will\ have \leq\ (2^{n+1}-1)-1\ edges.$$

$$Total\ number\ of\ Edges \leq 2^{n+1}-2\ edges.$$

<u>*For Depth Calculation*</u>

$$But\ for\ depth, we\ take\ edge(k) = n.$$

$$Again, for\ depth:$$

$$N(Nodes) = Edges(k) + 1(Root)$$

$$\Longrightarrow N = n + 1\ ,$$

$$Maximum\ Depth = Number\ of\ Edges = n.$$

$Based\ on\ Active\ stackframe$:

$$\therefore Total\ Auxiliary\ Space$$

$$= (\ Maximum\ Depth + 1) \times \begin{pmatrix} Space\ per \\ Recursive\ Call \end{pmatrix}$$

$$\left[\begin{array}{c}\boldsymbol{Here + 1\ for\ Root\ Node\ , which\ is\ also\ an\ active} \\ \boldsymbol{stack\ frame}\end{array}\right]$$

$$= \boldsymbol{\Theta}(\mathbf{n} + \mathbf{1}) \times \boldsymbol{\Theta}(\mathbf{1})$$

$$= \boldsymbol{\Theta}(\mathbf{n}) \times \boldsymbol{\Theta}(\mathbf{1})$$

$$= \boldsymbol{\Theta}(\boldsymbol{n})$$

$\therefore$ $\boldsymbol{Total\ Space\ Complexity}$:

$$\boldsymbol{Input\ Space + Auxiliary\ Space} = \boldsymbol{\Theta}(\mathbf{1}) + \boldsymbol{\Theta}(\boldsymbol{n})$$
$$= \boldsymbol{\Theta}(\boldsymbol{n}).$$